

Aperture — Database Design

Version: v1.0

Document Type: Database Design

Related Documents

- Domain Model
 - Low-Level Design
 - Technical Architecture
-

1. Purpose

This document defines how Aperture's business entities are persisted.

Unlike the Domain Model, which describes business concepts, this document focuses on storage design.

It answers questions such as:

- How is data stored?
- Why is it stored this way?
- How are relationships modeled?
- How do we maintain consistency?
- How do we scale as data grows?

The schema is intentionally optimized for clarity, correctness, and future evolution rather than premature optimization.

2. Database Philosophy

The database is the single source of truth for Aperture.

Every table should satisfy four principles:

Normalize first

Avoid unnecessary duplication.

Redundant data should only be introduced after identifying measurable performance bottlenecks.

Preserve relationships

Aperture is inherently relational.

Examples include:

- users ↔ reviews
- movies ↔ cast
- movies ↔ genres
- users ↔ lists
- users ↔ follows

The schema should make these relationships explicit.

Design for evolution

New features should extend the schema rather than replacing it.

Future requirements should influence naming and organization, but not lead to speculative tables.

Optimize with evidence

Indexes, caches, and denormalized views should be added only after profiling demonstrates a clear need.

3. Storage Strategy

Different categories of data have different storage requirements.

Data Type	Primary Store	Notes
User identities	PostgreSQL	Transactional
Movies & TV	PostgreSQL	Canonical metadata
Reviews	PostgreSQL	User-generated content
Lists	PostgreSQL	Ordered collections
Search index	OpenSearch (future)	Derived data
Cache	Redis	Non-authoritative

Data Type	Primary Store	Notes
Embeddings	PostgreSQL (pgvector initially)	AI features
Images	TMDb CDN / Object Storage (future)	Referenced, not embedded

Only PostgreSQL is considered authoritative.

Everything else is derived or cached.

4. Naming Conventions

Consistency is prioritized over brevity.

Guidelines:

- Singular entity names in code (`Movie` , `Review`)
- Plural table names (`movies` , `reviews`)
- Snake_case column names
- UUID primary keys
- `created_at` and `updated_at` timestamps on mutable entities
- Soft-delete timestamps (`deleted_at`) where recovery is important

5. Primary Entity Groups

The schema is organized into logical domains rather than feature folders.

Identity

- identities
- refresh_sessions

Users

- users
 - user_preferences
 - follows
-

Metadata

- movies
 - tv_shows
 - seasons
 - episodes
 - people
 - genres
 - collections
 - studios
 - movie_cast
 - movie_crew
 - tv_cast
 - tv_crew
-

Reviews

- reviews
 - ratings
 - review_likes
-

Lists

- lists
 - list_items
-

Streaming

- providers
 - availability
-

Notifications

- notifications
 - notification_preferences
-

Administration

- audit_logs

- feature_flags

This organization mirrors the domain boundaries established in the LLD.

6. Primary Key Strategy

Decision

UUIDs

Why?

Advantages:

- Globally unique
- Suitable for distributed systems
- Avoid sequential ID enumeration
- Easier future service extraction

Tradeoffs:

- Larger indexes
- Slightly slower inserts compared to integers

The benefits outweigh the costs for Aperture's expected scale.

7. Relationship Strategy

Relationships are modeled explicitly using foreign keys and junction tables.

Examples:

Movie ↔ Genre

Many-to-many

↓

movie_genres

Movie ↔ Person

Many-to-many

↓

movie_cast

movie_crew

User ↔ User

Many-to-many

↓

follows

List ↔ Movie

Many-to-many (ordered)

↓

list_items

Explicit junction tables preserve flexibility and make future metadata additions easier.

8. Normalization Strategy

The schema targets **Third Normal Form (3NF)** for transactional data.

Examples:

- Genres are stored once and referenced.
- People are stored once and linked to many titles.
- Providers are stored once and referenced by availability records.

Intentional denormalization is reserved for performance-critical scenarios and should be documented when introduced.

9. Data Lifecycle

Each major entity has a defined lifecycle.

Movie

Imported



Updated



Referenced



Archived (future)

Review

Created



Edited



Reported (optional)



Deleted (soft delete)

List

Created



Updated



Shared



Archived / Deleted

Understanding these lifecycles informs retention policies and indexing strategies.

10. Referential Integrity

Foreign key constraints should enforce consistency wherever practical.

Examples:

- A review cannot exist without a user.
- A list item cannot reference a nonexistent movie.
- A follow relationship requires two valid users.

Cascade behavior should be chosen deliberately. In many cases, soft deletion is preferable to cascading deletes to preserve historical records.

11. Indexing Philosophy

Indexes should support the most common access patterns.

Likely candidates include:

- User lookup by username
- Movie lookup by TMDb ID
- Review lookup by movie
- Review lookup by user
- List lookup by owner
- Notification lookup by recipient
- Availability lookup by movie and region

Composite indexes should be introduced based on observed query patterns rather than speculation.

12. Soft Deletes

Soft deletion is preferred for entities where recovery, moderation, or auditing is important.

Examples:

- Reviews
- Lists
- User accounts (deactivation)
- Notifications (optional)

Reference data such as genres or providers typically does not require soft deletion.

13. Transactions

Transactions should protect operations that modify multiple related entities.

Examples:

- User registration (Identity + User)
- Review creation (Review + Rating)
- List creation with initial items
- Follow creation with count updates

Transactional boundaries should remain as small as possible while preserving consistency.

14. Migration Strategy

Schema evolution is managed through version-controlled migrations.

Guidelines:

- Small, incremental changes
- Backward-compatible where practical
- Rollback plans for risky migrations
- No manual production schema changes

Every migration should be reviewed alongside the corresponding application changes.

15. Growth Strategy

The schema is designed to evolve without disruptive rewrites.

Examples:

- Read replicas for scaling reads
- Partitioning large activity tables
- Materialized views for analytics
- Separate search indexes
- Dedicated vector storage if needed

Growth should be driven by measured bottlenecks rather than anticipated scale.

16. Backup & Recovery

Production databases should support:

- Automated backups
- Point-in-time recovery
- Backup verification
- Restore testing

Operational readiness includes proving that backups can be restored, not merely creating them.

17. Security

Sensitive data should be minimized.

Examples:

- Passwords are stored only in the Authentication domain as hashes.
- Secrets are never stored in application tables.
- Personally identifiable information is limited to what the product requires.

Database access follows the principle of least privilege.

18. Interview Discussion Points

Be prepared to explain:

- Why choose UUIDs over auto-increment IDs?
 - Why normalize instead of embedding related data?
 - When would you intentionally denormalize?
 - Why keep PostgreSQL as the source of truth?
 - How would you scale the database without redesigning the schema?
 - How do domain boundaries influence table organization?
-

19. Summary

Aperture's database design emphasizes strong relational modeling, clear ownership, and evolutionary growth.

By separating business concepts from persistence decisions and aligning the schema with the platform's modular architecture, the design remains maintainable for a single developer while providing a credible path toward larger-scale production systems.

The following sections (to be completed in subsequent parts) will define:

- Entity-by-entity schemas
- Relationship diagrams
- Index recommendations
- Query patterns
- Partitioning strategies
- Data retention policies
- Example migrations